Cristian Alarcon

PROYECTO LINEDU

Proyecto II de SIstemas operativos II

Lucas Soto

Etapa II: Estudio de Factibilidad (SDLC)

1. Factibilidad Técnica

Este análisis evalúa si el proyecto puede ponerse en marcha y mantenerse con los recursos tecnológicos y el equipo de desarrollo actual, contemplando la viabilidad de la implementación y su posterior funcionamiento.

- **¿Puede realizarse con el equipo actual?** Sí. Al haber seleccionado un enfoque de **Simulación Basada en Bloques Dinámicos** (sin virtualización ni contenedores Docker corriendo en tiempo real), el entorno de desarrollo no requiere servidores de alta gama. Computadoras estándar con procesadores de gama media y 8-16 GB de RAM son perfectamente capaces de ejecutar el entorno de compilación de Expo, la base de datos local y el servidor de desarrollo API REST.
- **¿Existe personal disponible para desarrollarlo?** El desarrollo técnico será llevado a cabo de forma íntegra por el alumno en el rol de programador Full-Stack, cubriendo tanto la interfaz móvil (Front-end) como el diseño y lógica de las bases de datos y validaciones (Back-end).
- **¿Se necesita nueva tecnología?** No se requiere infraestructura ajena o inaccesible. Se trabajará sobre tecnologías open-source y de amplio soporte:
    - **Front-end:** Expo y React Native (TypeScript), que permiten compilar de forma nativa para plataformas Android e iOS sin duplicar bases de código.
    - **Back-end:** Node.js con Express para construir la API RESTful.
    - **Base de Datos:** PostgreSQL o MongoDB para almacenar usuarios, rachas e históricos de desafíos.
    - **Validación:** El motor se basa en patrones lógicos de **Expresiones Regulares (Regex)**, un estándar nativo del lenguaje que no añade dependencias pesadas ni costos extras.

Dictamen Técnico: FACTIBLE. El diseño arquitectónico está optimizado para consumir el mínimo de recursos computacionales, lo que garantiza que la app puede ser desarrollada y mantenida sin problemas de infraestructura.

2. Factibilidad Económica

Para justificar la inversión del proyecto frente al beneficio económico que generará, se establece una estructura de costos (fijos y variables) y una estrategia de monetización basada en el mercado regional y nacional.

A. Estructura de Costos (Estimado Anual)

Al tratarse de un desarrollo de software independiente, se consideran costos de infraestructura en la nube y licencias esenciales.

Tipo de Costo	Concepto / Descripción	Costo Mensual	Costo Anual Total
Costo Fijo	Hosting de API y Base de Datos (Render / Supabase / AWS Starter)	$15.000 ARS	$180.000 ARS
Costo Fijo	Cuenta de Desarrollador Google Play Store (Pago único de $25 USD)	—	$35.000 ARS
Costo Fijo	Mantenimiento de Dominio Web (`.com.ar` para landing page/diplomas)	—	$15.000 ARS
Costo Variable	Consumo extra de Base de Datos / Tráfico de red (Escalabilidad de usuarios)	Tarifa base	$50.000 ARS
TOTAL ESTIMADO	**Inversión requerida para mantener el sistema operativo**	—	**$280.000 ARS**

B. Estrategia de Ventas y Fijación de Precios

El modelo de negocio adoptado es **Freemium**. La aplicación será de descarga gratuita con acceso a las lecciones básicas de Linux. La monetización se define mediante dos vías principales:

1. **Suscripción Premium ("LinEdu Plus"):** Acceso a explicaciones detalladas, desafíos avanzados (redes, scripts de Bash) y la emisión automática del Diploma de Competencia tras alcanzar los 30 días de racha.
2. **Precio Definido:** El abono mensual de la suscripción se fija en **$3.500 ARS** (un precio sumamente accesible, equivalente al valor de un café comercial, diseñado para estudiantes y profesionales de tecnología en Argentina).

C. Proyección de Ventas (Período: 1 Año)

La estimación se calcula asumiendo una tasa de conversión conservadora del 3% de usuarios activos mensuales a la modalidad Premium, basada en estadísticas promedio de aplicaciones educativas móviles (como Duolingo o Mimo).

- **Meta de Usuarios Activos (Mes 1 al 12):** Alcanzar una base estable de 3.000 usuarios en el país, consolidando 90 suscriptores activos Premium al mes a partir del segundo semestre.

Proyección Financiera Anual:
Meses 1 - 3 (Lanzamiento y Ajustes): ~ 20 Suscriptores Promedio = $210.000 ARS
Meses 4 - 6 (Crecimiento Orgánico): ~ 50 Suscriptores Promedio = $525.000 ARS
Meses 7 - 12 (Madurez en Mercado): ~ 90 Suscriptores Promedio = $1.890.000 ARS

INGRESO BRUTO ANUAL ESTIMADO: $2.625.000 ARS
(-) COSTOS TOTALES OPERATIVOS: -$280.000 ARS

BENEFICIO NETO ESTIMADO (Año 1): $2.345.000 ARS

Dictamen Económico: FACTIBLE. El punto de equilibrio se alcanza en los primeros meses de distribución masiva. Los ingresos estimados superan ampliamente los bajos costos operativos que requiere la arquitectura de simulación planteada.

3. Factibilidad Operativa

La factibilidad operacional determina si el sistema será verdaderamente utilizado, si existe un mercado receptivo y si se presentará resistencia al cambio por parte de los usuarios.

- **¿El sistema será utilizado?** Sí. Existe una alta demanda insatisfecha de herramientas educativas interactivas de Linux en el sector hispanohablante. Estudiantes de escuelas técnicas, institutos superiores, universidades y entusiastas de la ciberseguridad requieren incorporar estos conocimientos en su portafolio técnico de manera ágil.
- **¿Existirá resistencia al cambio?** La resistencia al cambio en este escenario es prácticamente nula, ya que la aplicación no viene a reemplazar un sistema laboral obligatorio, sino a resolver una frustración existente. Al eliminar las barreras del teclado móvil mediante el sistema de bloques interactivos, los usuarios encuentran una experiencia de aprendizaje lúdica y fluida, lo que incrementa la retención en lugar de generar rechazo.
- **Capacitación y Roles:** El usuario objetivo posee un nivel básico de manejo de smartphones, por lo que la curva de aprendizaje de la aplicación es inmediata. Las mecánicas de juego están inspiradas en interfaces familiares ya validadas globalmente por el mercado pedagógico (estilo Duolingo).

Dictamen Operativo: FACTIBLE. La aplicación cubre un vacío didáctico real en la formación tecnológica. La experiencia de usuario diseñada disuelve la fricción sintáctica tradicional de la consola, garantizando una alta aceptación por parte de los estudiantes.

Tabla de casos de uso.

ID	Caso de Uso	Actor(es)	Descripción del Comportamiento Técnico
CU-01	**Consumir Desafío Dinámico**	Usuario / Sistema	El usuario ingresa a la sección de aprendizaje. La aplicación móvil (Expo/React Native) realiza una petición GET de forma asíncrona a la API REST del backend. El servidor extrae un desafío desde la base de datos (PostgreSQL/MongoDB), toma la lista de componentes de la sintaxis correcta, inyecta los distractores semánticos directos configurados y desordena el pool de bloques antes de retornar la carga útil en formato JSON a la interfaz del dispositivo cliente.
CU-02	**Construir Comando interactivo**	Usuario	El usuario visualiza el enunciado en lenguaje natural, la línea de comandos simulada vacía y el pool de botones desordenados. El usuario interactúa tocando los bloques interactivos de texto en un orden sintáctico determinado. La interfaz actualiza el estado local (React State) moviendo los elementos seleccionados hacia la línea de comandos vacía en tiempo real, bloqueando el teclado táctil nativo del smartphone para evitar errores de tipeo y frustración.
CU-03	**Validar Sintaxis del Comando**	Usuario / Backend	Al presionar el botón de confirmación, la app del cliente envía un arreglo ordenado de strings al endpoint de validación. El backend recibe la estructura de datos, la concatena dinámicamente insertando espacios en blanco (`" "`) y evalúa el string resultante

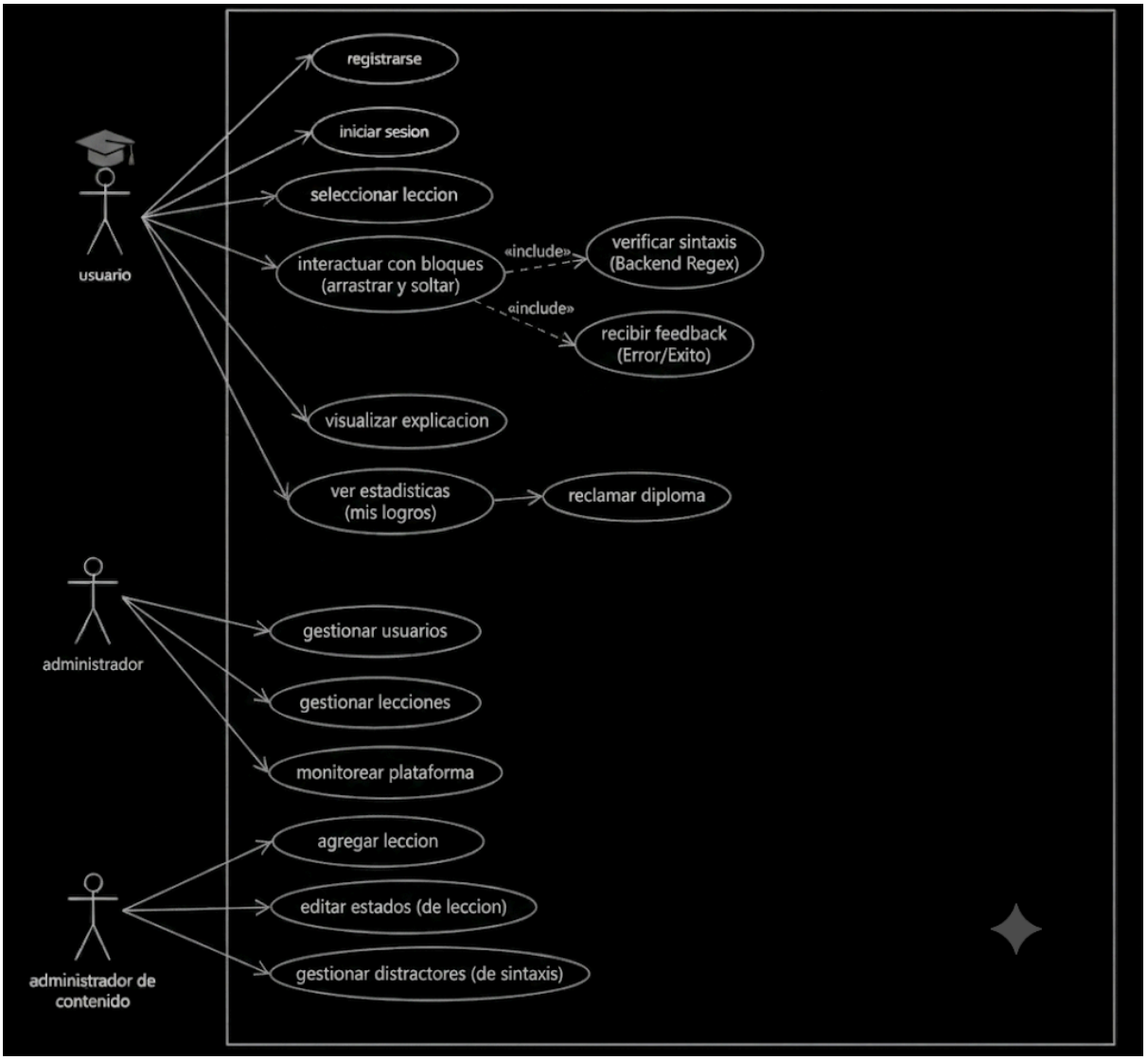
ID	Caso de Uso	Actor(es)	Descripción del Comportamiento Técnico
			aplicando un test basado en expresiones regulares (Regex). Esto otorga flexibilidad computacional permitiendo la variación posicional de flags u opciones equivalentes (ej. `ls -la` o `ls -al`) sin requerir emulación virtualizada. El sistema retorna un estado inmediato de aprobación o error semántico.
CU-04	**Actualizar Motor de Rachas (Streaks)**	Servidor (Backend)	Tras una validación sintáctica exitosa (CU-03), el middleware del servidor invoca de forma interna la lógica de gamificación horaria. Compara la marca de tiempo de la base de datos de la última lección aprobada contra el timestamp UTC actual del servidor de aplicaciones. Si la ventana temporal se ubica entre las 24 y las 48 horas, incrementa el contador de racha en +1 día. Si la diferencia es menor a 24 horas, registra el acierto sin alterar el contador diario. Si excede las 48 horas, se penaliza el abandono reseteando la racha a un valor base de 1.
CU-05	**Generar Diploma Criptográfico**	Sistema (Módulo Crypto)	Este caso de uso se ejecuta automáticamente como una extensión posterior a la actualización de rachas (CU-04). Al momento en que el contador de días ininterrumpidos en el backend alcanza estrictamente el valor de 30 días, el servidor interrumpe la respuesta estándar para invocar un servicio criptográfico interno. Este servicio genera un identificador universal único (UUID) inmutable que actúa como credencial y aval del diploma de competencia en fundamentos de Linux, almacenándolo en el repositorio persistente de certificaciones para su libre acceso vía web o app.

Resumen de los Casos de Uso del Documento:

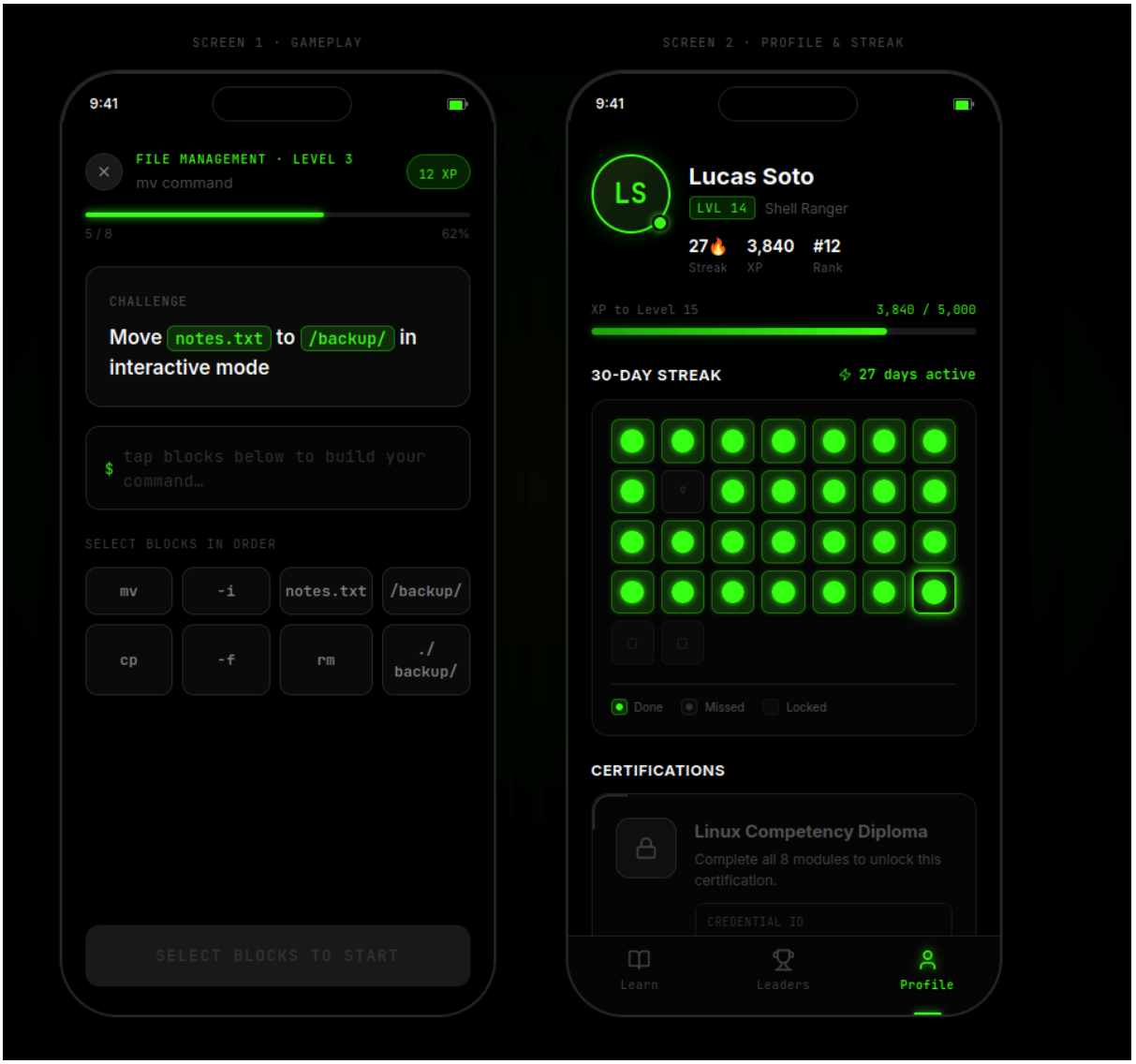
La tabla detalla las interacciones críticas del sistema, cada una con su identificador único y una explicación sobre su comportamiento técnico en base a la arquitectura definitiva:

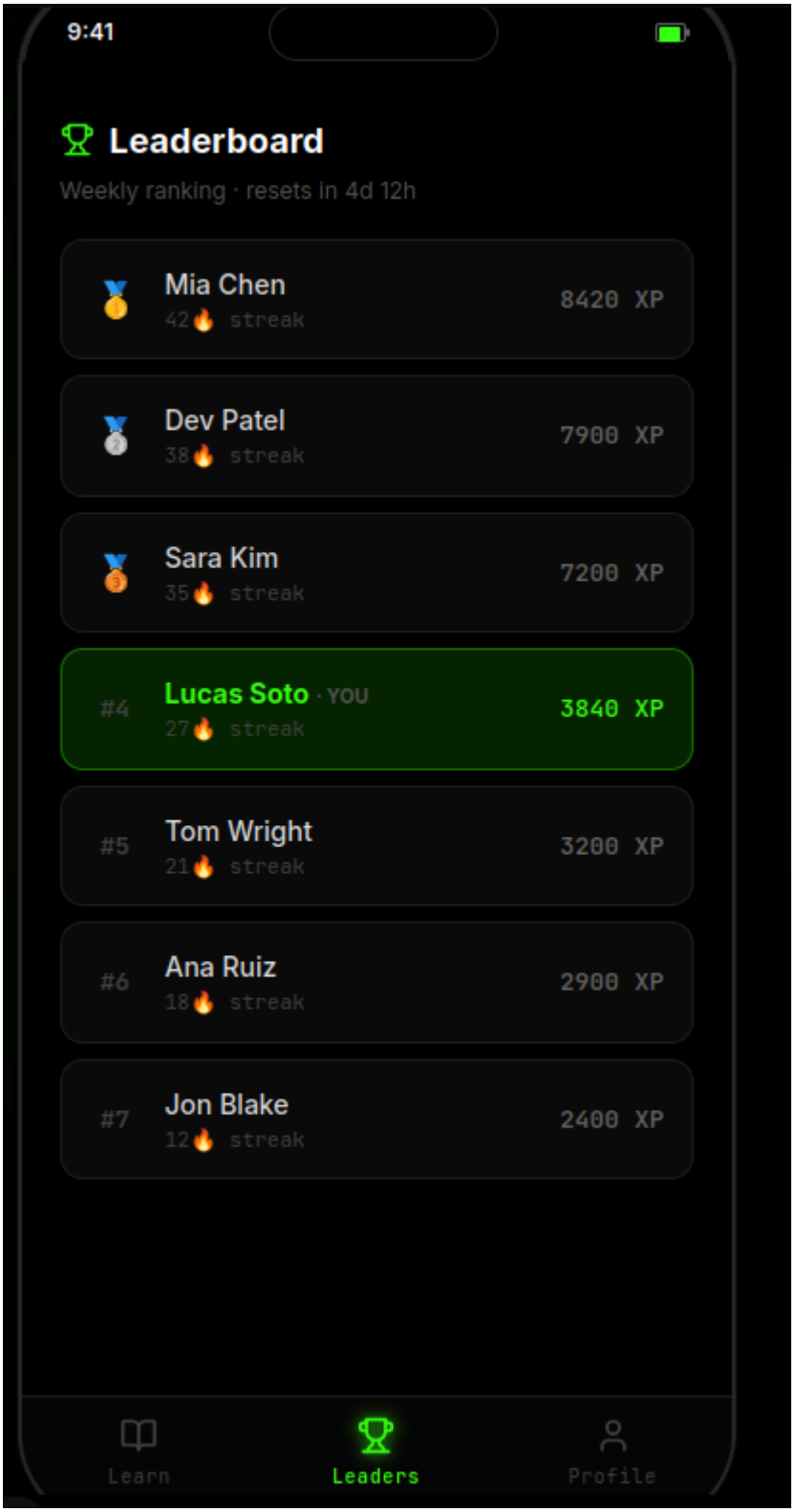
1. **CU-01: Consumir Desafío Dinámico:** Cómo el front-end (Expo) solicita lecciones de forma asíncrona a la API REST y el backend junta los bloques válidos con los **distractores semánticos directos** para enviar la carga aleatoria.
2. **CU-02: Construir Comando Interactivo:** Detalla la interacción del usuario tocando bloques táctiles en la interfaz gráfica, actualizando los estados de React y manteniendo oculto por completo el teclado del móvil.
3. **CU-03: Validar Sintaxis del Comando:** Explica la recepción en el backend del array ordenado, su concatenación con espacios vacíos y la ejecución del test con **Expresiones Regulares (Regex)** para habilitar la flexibilidad sintáctica (ej: banderas como `ls -la` y `ls -al`).
4. **CU-04: Actualizar Motor de Rachas (Streaks):** Cómo actúa el backend comparando la fecha de resolución con el último acierto en marcas de tiempo **estrictamente UTC del servidor**, validando la ventana crítica de 24-48 horas para sumar o reiniciar la racha.
5. **CU-05: Generar Diploma Criptográfico:** El disparador automático que se ejecuta en el servidor cuando un alumno alcanza la meta matemática de 30 días seguidos de racha, usando el módulo criptográfico nativo para firmar un **UUID único** e inmutable.

Diagrama de flujo:
Este es el diagrama de flujo de Linedu, teniendo en cuenta el usuario, administrador y administrador de contenido.



Pantallas del programa




9:41
Leaderboard
Weekly ranking · resets in 4d 12h
Mia Chen
42 streak
8420 XP
Dev Patel
38 streak
7900 XP
Sara Kim
35 streak
7200 XP
#4
Lucas Soto · YOU
27 streak
3840 XP
#5
Tom Wright
21 streak
3200 XP
#6
Ana Ruiz
18 streak
2900 XP
#7
Jon Blake
12 streak
2400 XP
Learn
Leaders
Profile